

F2

A window-and-scope modular sequencer — user manual

F2 project July 2026

v2.0

TABLE OF CONTENTS

01	About this document	19	Workflow and the project
02	The paradigm: modulation by windows and scopes	20	The canvas: blocks and the tree
03	Signal-path architecture: channels	21	The palette and presets
04	The scope hierarchy	22	The audio engine and live state
05	Time: phase, loops, the phase tree	23	The interface: overview
06	Windows	24	Modulator cards
07	Modulators: overview and kinds	25	The scope bus and the value stack
08	Forms (curves)	26	The scenes panel and macros
09	Func processors	27	The inspector, LockGraph, command palette
10	Routing: parameter or inlet	28	The effects rack
11	Value composition	29	Harmony
12	Reverse patching: listen	30	Writing synthesizers
13	Windows as signals: clocks and onsets	31	Writing effects
14	Randomness: rand, dice, variants	32	Saving and compatibility
15	span: multi-loop forms	33	Diagnostics
16	Macro channels	34	Recipes
17	Cross-row chains	35	Reference tables
18	Reactivity and live editing	36	Glossary

01 — ABOUT THIS DOCUMENT

This manual describes F2 — a sequencer-synthesizer in which parameter modulation is organized as a modular system governed by **time windows** and a **scope hierarchy**. The document covers the paradigm, the data model, the semantics of the modulation engine, and every element of the interface.

The reader is assumed to command basic DSP notions (sampling, envelopes, LFOs, sample-and-hold, quantization) and to be familiar with modular-synthesis practice (patching, CV, attenuverters, clocks, dividers). Terms at that level are used without definition.

The manual consists of three parts. The first describes the paradigm and the engine semantics — read it sequentially; the interface cannot be understood without the model it displays. The second describes the interface element by element and is organized as a reference: each figure comes with a numbered list of elements, and numbers in the text refer to numbers on the figures; this part also contains the chapters on the project, the effects rack, harmony, and authoring synthesizers and effects. The manual closes with diagnostics, recipes for typical patches, reference tables, and a glossary.

Notation conventions

- Interface element names are written exactly as displayed on screen: `ramp`, `lock-ramp`, `+ listen`, `→ target`.
- Channel paths and selectors are set in monospace: `m:1`, `~win`, `b:20/c:3/#grPos`.
- A number in parentheses — for example (3) — refers to the numbered element of the current figure.
- Phase ranges are written as $t \in [0, 1)$: 0 included, 1 excluded.
- A **row** is a horizontal lane of the scenes panel with its own loop length.

Every value, form and choice in F2 is deterministic. Wherever the

text says "random", read "pseudo-random, reproducible from a known seed". This is a foundational property of the system, not an implementation detail; see the chapter on randomness.

02 — THE PARADIGM: MODULATION BY WINDOWS AND SCOPES

The difference from classic modular synthesis

In a hardware modular synthesizer a module runs continuously: an LFO oscillates for as long as it is powered, and the question "when does the modulation act" is answered by external means — VCAs, gates, sequencers. Time is a resource external to the patch.

F2 inverts that relation. Here **time is a structural property of the patch itself**: every modulator belongs to a node of the temporal structure (a scope), and that node defines a **window** — the interval of loop phase inside which the modulator exists. Outside the window the modulator's output signal is absent as an object, and everyone who listened to it automatically falls back to their own values.

Three practical properties follow:

1. **A form always lives a complete life cycle.** A ramp assigned to a scope travels from → to in exactly the duration of the scope's window — however short that window is. The same ramp on a whole-loop window is a slow sweep; on a 1/16-loop window it is a click-gesture. Forms are described in phase units, not in absolute time.
2. **Modulation never "sticks".** The classic problem of a released gate — the parameter stuck at the last CV value — is absent by construction: closing the window removes the contribution, and the target parameter returns to its base (or to the contributions of senior scopes).
3. **The structure of time and the structure of modulation are one object.** A branching of the sequence (a random branch choice) is simultaneously a branching of the modulation patch: modulators of the unchosen branch do not sound, because their windows never opened.

The second dimension: scopes

Orthogonal to time runs the **scope hierarchy** — the levels at which values and modulators are declared: preset, tree node, block, cell. A scope defines two things: the **window** (when the modulator acts) and the **rank** (who wins a conflict over one parameter). The conflict-resolution rule is single and universal: **the deeper (junior) scope overrides the more senior one**. It acts identically for base parameter values, for modulation layers, and for several writers of one inlet.

The third dimension: addressing

Any modulator is a module with inputs (inlets) and one output. The output is routed in one of two ways:

- **to a parameter name** — "the parameter in the general sense": the contribution lands as a layer on all cells within the scope's range whose presets have that parameter; conflicts are resolved by rank;
- **to a concrete inlet of a concrete modulator** — point addressing by a stable identifier: the output becomes a signal inside another module's input (func stacks, phase control, clocks).

Both ways are available to every kind of modulator. The distinction "are we modulating the parameter in general, or a concrete input of a concrete node" is one of the two or three key distinctions the whole system is built upon; the interface reproduces it as two adjacent controls on every card (see the chapter on cards).

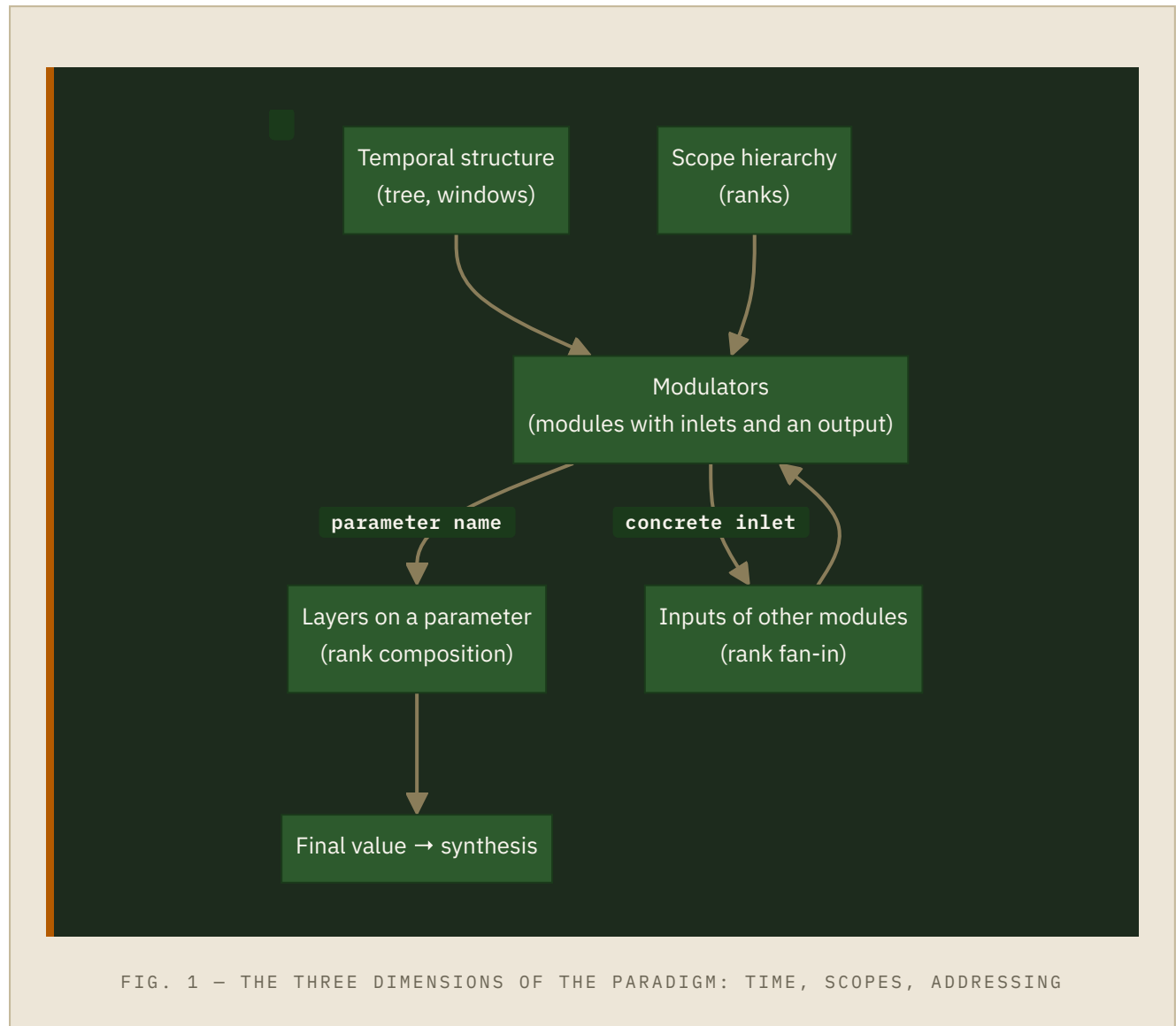


FIG. 1 — THE THREE DIMENSIONS OF THE PARADIGM: TIME, SCOPES, ADDRESSING

03 — SIGNAL-PATH ARCHITECTURE: CHANNELS

Everything is a channel

The internal modulation state is organized as a flat space of **channels** addressed by paths — by analogy with a file system. A channel is a named numeric value that may exist or not exist. Channel existence is a semantically meaningful state: reading an absent channel returns "no value", and the reader is obliged to use its own fallback.

Channel kinds:

CHANNEL	KIND	EXISTENCE
<code><scope>/#<param></code>	base value	while a lock/knob is declared
<code><scope>/@<uid></code>	modulator output	EXACTLY while the scope window is open
<code><cell>/=<param></code>	parameter result	always (delivered to synthesis)
<code><scope>/~win</code>	scope window phase	EXACTLY while the window is open
<code>m:<k>/#value</code>	macro knob	always (global)

- `#`-channels are base values: a knob position, a param-lock value. Turning a knob of any scope is a write of one channel; nothing is recompiled.
- `@`-channels are live modulator outputs. Their presence is the very mechanism of the window contract: closing the window deletes the channel.
- `=`-channels are the composition result for a concrete cell; they are what gets delivered to the synthesizer. They exist continuously for **all** cells of the structure (including unchosen branches), which is why knobs are heard instantly regardless of which branch the sequencer is currently playing.
- `~win`-channels are window phase as a signal (chapter on windows as signals).
- `m:`-channels are the global macros (chapter on macros).

The base-value cascade

Reading a cell's base value for a parameter is a climb up the path from the cell: "the nearest existing channel wins".

```
1 c:3/#grPos → b:20/#grPos → t:n106/#grPos → ... → p:<preset>/#grPos
```

This is exactly the scope hierarchy expressed in addresses: a cell lock overrides a block lock, the block lock overrides the container, and all of them override the preset base. Deleting a lock reactively "opens" the parent level.

The shared space of rows

All rows (scene lanes) live in **one** channel space. Consequences: the preset layer is honestly global (the preset is one — its channels are one for all rows); a modulator of one row can be heard in another row through a channel (chapters on cross-row chains and reverse patching); macros are visible to everyone.

04 — THE SCOPE HIERARCHY

Four levels

SCOPE	RANK	WINDOW	TYPICAL USE
preset	0	window of the cell playing it	sound base; parameter "gates"
tree node	1+	window of the tree node	section modulation, branches
block	n	window of the block in the tree	phrase-wide modulation
cell	n+1	window of one cell (slot)	per-step gestures, slices

Rank grows with depth: the preset is the most senior (base) layer, the cell the most junior. Inside the tree a deeper container is junior to a shallower one. The exact rule: rank equals the depth of the scope's path; the preset has rank 0.

- **Preset** — a global object (one for all rows). A preset modulator acts wherever a cell of this preset plays; its window is that cell's window. Preset func processors naturally form parameter "gates" — stationary nodes into whose inlets modulators of any scope are patched (see Routing).
- **Tree node (container)** — the window equals the share of phase the node occupies in its parent. A modulator on a branch container acts exactly when the branch is chosen and playing.
- **Block** — a strip of cell slots; the window is the block's interval in the tree. A block modulator "sees" all cells of the block.
- **Cell** — the minimal element; the window is its slot inside the block. For a block of height $h > 1$ every row of the block is a parallel sub-block in time; a cell is addressed by coordinates (x, y).

The two meanings of "parameter"

A synthesizer parameter (say `grPos`) exists in the system in two roles, and they must be distinguished:

1. **Name-assignment.** A modulator with a scope and a parameter name lays a layer "on the name": the layer applies to every sounding cell within the scope whose preset has that parameter. This is the "param lock" semantics inherited from step sequencers, generalized to the hierarchy.
2. **Input of a concrete node.** The same parameter as a field of a concrete modulator (a ramp's `from`, a quantizer's `value`) is a separate object with its own address; it is driven point-to-point ($\rightarrow$ target) or by a subscription (listen).

A numeric cascade example

Let parameter `grPos` be declared as: preset base 0.20; a block lock 0.40; a lock on cell c:3 — 0.90. Then:

- cell c:3 reads 0.90 (its own channel exists — it wins);
- the other cells of the block read 0.40 (the block channel);
- cells of other blocks of the same preset read 0.20 (the base).

Deleting the cell lock immediately (with no recompilation) opens the block value: c:3 starts reading 0.40. Deleting the block lock opens the base. The mechanism is one and the same — existence/absence of a channel in the cascade.

05 – TIME: PHASE, LOOPS, THE PHASE TREE

Global phase

The audio engine (SuperCollider) provides a single time primitive — a linear global beat extrapolated from an anchor (bpm, start moment). There are no bars, grids or quantization at this level.

A row is a stream

Every row of the scenes panel is an independent stream with its own loop length `dur` (in beats). From the global beat the stream derives:

- `iter = floor(beat / dur)` — the iteration (loop) number;
- `phase = frac(beat / dur)` — the loop phase, monotonic $0 \rightarrow 1$.

Rows with different `dur` are polymetric by construction. Scene switching inside a row happens on iteration boundaries.

The tree

The content of a scene is a tree of containers and block references. Container operators:

OPERATOR	NAME	CHILD-WINDOW SEMANTICS
<code>~q seq</code>	sequence	equal split of the parent window, in order
<code>~w wseq</code>	w-seq	split proportional to child weights
<code>~c par</code>	parallel	every child gets the WHOLE parent window
<code>~r rand</code>	random	one branch per iteration, equal odds
<code>~rw wrand</code>	w-random	one branch per iteration, odds by weight

The tree is compiled **in advance** into a set of **variants**: a variant is a complete resolution of all `rand` / `wrand` choices. The body of a variant is a flat sequence of **segments**; a segment is a cell with an interval $[t_0, t_1]$ of loop phase and with the windows of every scope on its path already computed (container windows, block window, the cell's own). At runtime the tree is not traversed: active segments are found by binary search over phase.

The weight of a variant is the product of the normalized weights of its choices. Full enumeration is bounded by a constant (64 variants); for larger spaces the concrete choice vector is resolved lazily with a cache — the semantics are identical.

An empty cell (a pause) occupies a time slot but produces no

segment: within its interval the cell's scope window does not exist, and neither sound nor modulation of that slot exists.

A numeric window example

A row with loop `dur = 32` beats. The tree: `par( block A (16 cells) || wseq( wrand(X:3, Y:1), block B ) with weights 1:3 )`.

- The window of block A is the whole loop $[0, 1)$: each of its cells i gets the window $[i/16, (i+1)/16)$, i.e. 2 beats.
- `wseq` with weights 1:3 splits the loop: first child $[0, 0.25)$, second $[0.25, 1)$.
- `wrand(X, Y)` occupies the whole window of the first child: the branch chosen this iteration gets the entire $[0, 0.25)$ — 8 beats; the unchosen one gets no window at all.
- Block B occupies $[0.25, 1)$ — 24 beats.

There are two variants (one per `wrand` branch): the variant with X has weight 0.75, with Y — 0.25. A modulator declared on the container of branch X has the window $[0, 0.25)$ in variant X and does not exist in variant Y.

Variant choice

On every iteration one variant is chosen — by a deterministic weighted roll from the tuple `(row, scene, iter, epoch)`. The choice is known in advance to every side of the system (interface, engine, audio player) with no message exchange; the audio player additionally receives the choices of the current and the next iteration as explicit messages so that demand chains can run ahead. `epoch` is the counter of structural scene restarts (chapter on reactivity); incrementing it "re-rolls the dice".

Branch weights are read at roll time from the channels `<branch>/#weight` when such channels exist, otherwise from the static tree weights. A computed iteration choice is frozen: what has already been announced to the audio player is not re-played when a knob turns mid-loop.

06 — WINDOWS

Definition

The **window of a scope** is the interval $[w0, w1)$ of loop phase computed at variant compilation. A cell's window is its slot; a block's window is its interval; a container's window is its share of the parent; a preset's window is the window of the cell in which the preset is currently playing.

Window phase is the linear value $w_p \in [0, 1)$ equal to $(\text{phase} - w0)/(w1 - w0)$. All modulator forms are evaluated from the window phase of their scope — this is precisely the "complete life cycle" mechanism.

The window contract

The formulation that defines the entire system:

A modulator acts exactly inside the window of its scope. Outside the window its contribution is absent, its output channel `@uid` is deleted from the channel space, and any readers of that channel use their own fallback values. Inside the window the form runs a complete cycle from start to finish.

The contract extends to every signal-delivery mechanism: the layer on a parameter disappears; writing into a target inlet stops; a channel subscription (listen) sees the channel's absence and falls back to the inlet's own constant. "Frozen" values do not exist in the system.

There is one deliberate exception: `=`-channels (a cell's parameter result) exist always and for all cells — base values are delivered to synthesis continuously so that knobs act instantly regardless of windows and branch choices. Windows gate **modulation only**.

07 — MODULATORS: OVERVIEW AND KINDS

The module model

Every modulator is described by: kind, owner scope, inlets, output, output routing, and combine mode. An inlet holds a constant (the knob position) and, optionally, sources: in-patch writers (→ target of other modules) and channel subscriptions (listen). The output is published to the `@uid` channel (inside the window) and routed either to a parameter name or into a target inlet.

Kinds

KIND	CARD	FUNCTION
<code>const</code>	<code>param</code>	constant on a parameter of its scope (param lock)
<code>source</code>	<code>ramp</code>	form of window phase: from → to over the window
<code>func</code>	<code>func</code>	stateful value processor (9 processors)
<code>chain</code>	<code>lock-ramp</code>	source whose output writes into a target inlet

The `ramp` / `lock-ramp` distinction is historical and survives in the interface as a convenient "defaults to a parameter" / "defaults to an inlet" split; semantically both are sources, and both can be routed either way (see the chapter on routing).

Inlets by kind

KIND	INLETS
<code>const</code>	<code>value</code>
<code>source</code>	<code>from</code> , <code>to</code> , <code>rate</code> , <code>peak</code> , <code>susl</code> , <code>speed</code> , <code>hold</code> , <code>morph</code> , <code>phase</code> , <code>seed</code>
<code>func</code>	<code>value</code> , <code>amount</code> , <code>clock</code>

Every inlet is modulatable: it can be driven by another modulator (a point write) or by a channel (a subscription). The `phase` and `seed` inlets are detailed in the chapters on windows-as-signals and randomness; `clock` is the trigger input of the `tsh`/count processors.

08 — FORMS (CURVES)

A form defines the value of a source modulator as a function of window phase $t \in [0, 1)$ and the from/to bounds. Each form with its parameters is given below. Parameters are read from the inlets of the same names.

Continuous

- **lin** — linear interpolation from $\rightarrow$ to.
- **exp** — exponential: $\text{from} \cdot (\text{to}/\text{from})^t$; from = 0 is substituted with 0.001. Suitable for frequency and time parameters.
- **log** — a saturating curve $\text{from} + (\text{to} - \text{from}) \cdot (1 - (1 - t)^3)$: fast start, smooth approach to to.
- **sin** — cosine ease-in-out between from and to (half a cosine period).
- **env** — a four-segment envelope with fixed phase breakpoints: attack 0–0.15 (from $\rightarrow$ peak), decay 0.15–0.4 (peak $\rightarrow$ susl), sustain 0.4–0.7 (susl), release 0.7–1 (susl $\rightarrow$ from). Parameters: **peak** (defaults to to), **susl** (defaults to the midpoint). Segment proportions are fixed relative to the window.
- **drift** — wandering around the middle of the range: the sum of two incommensurate sines with coefficients 0.3 and 0.2; the **speed** parameter sets the density. It does not progress from from to to; from/to only set the range.

Oscillating

- **lfo** — a morphable oscillator: **rate** periods per window; **morph** continuously drives the shape sine (0) $\rightarrow$ triangle (0.5) $\rightarrow$ square (1); all three are phase-aligned. from/to is the swing.

Stepped

- **step** — a staircase of N equal steps from from to to; $N = \text{hold}$ (minimum 2). The value of step i is $\text{from} + (\text{to} - \text{from}) \cdot i / (N - 1)$.
- **slew** — the same staircase, but the edge of every step is smoothed with a cosine transition over the first quarter of the step.

Stochastic

- **rand** — N steps per window ($N = \text{rate}$, minimum 4); the value of each step is a hash of the step number. Essential: **rand does not depend on the iteration** — the pattern is identical every loop. It is the "fixed random picture" form.
- **dice** — N independent rolls per window ($N = \text{rate}$, minimum 1), each roll a deterministic function of (iteration seed, step number). Unlike rand the pattern **re-deals every iteration**, remaining repro-

ducible and consistent across all components of the system. The `seed` inlet replaces the iteration seed (chapter on randomness). With `span` > 1 the pattern stretches and re-deals once per span iterations (chapter on span).

The practical rule of choice: `rand` — when you want a picture that

repeats every cycle (a stable "programmed" pattern); `dice` — when you want a fresh but reproducible deal on every loop (a live breakbeat slicer).

Exact formulas

For $t \in [0, 1)$, $s = \text{to} - \text{from}$:

FORM	VALUE	FORMULA
<code>lin</code>		<code>from + s · t</code>
<code>exp</code>		<code>from · (to/from)^t</code> (from = 0 → substituted 0.001)
<code>log</code>		<code>from + s · (1 - (1-t)^3)</code>
<code>sin</code>		<code>from + s · (0.5 - 0.5 · cos(πt))</code>
<code>lfo</code>		<code>from + s · osc((t · rate) mod 1, morph)</code>
<code>step</code>		<code>from + s · [t · N] / (N-1)</code> $N = \text{steps} \geq 2$
<code>slew</code>		like <code>step</code> , edge cosine-smoothed over the first quarter step
<code>drift</code>		<code>from + s · (0.5 + 0.3 · sin(t · v · π) + 0.2 · sin(t · v · 2.3π + 1))</code>
<code>rand</code>		<code>from + s · hash([t · N])</code> $N = \text{rate} \geq 4$; hash independent of iter
<code>dice</code>		<code>from + s · roll(seed, [t · N])</code> $N = \text{rate} \geq 1$; seed from the iteration
<code>env</code>		piecewise: 0–0.15 attack, 0.15–0.4 decay, 0.4–0.7 sustain, 0.7–1 release

`osc(φ, m)` is the phase-aligned sine→triangle→square morph ($m = 0 / 0.5 / 1$); `roll` is a 32-bit avalanche mix of (seed, step number), uniform on $[0, 1)$.

Preview equals engine

Form thumbnails on the cards and the preview at the bottom of a card are computed by the same mathematics as the engine (shared code) — the preview is not an approximation but the exact trajectory. For `dice` the preview uses seed zero: the picture illustrates the character of the steps but does not match the concrete deal of an iteration.

09 – FUNC PROCESSORS

Func is a stateful node: on every tick it takes the input value (the `value` inlet), applies the processor and produces the result. The processor's parameter is the `amount` inlet; trigger processors use the `clock` inlet. The processor's lo/hi range equals the range of the carrier parameter (from the session's range map); for a func routed into a foreign inlet the carrier parameter serves only as a reference range.

The time unit of the processors is the **beat** (a fraction of tempo), not the second: time constants scale with BPM.

PROCESSOR	PARAMETER (AMOUNT)	BEHAVIOUR
lag	time, beats	one-pole smoothing towards the input
slew	speed, units/beat	rate limiting of change
quantize	levels N (2...32)	snap to N equal levels in [lo, hi]
s&h	frequency, 1/beat	periodic sampling of the input
fold	gain (1...8)	wavefold around the range centre
tsh	–	sample the input ON the clock EDGE
cmp	threshold	input > threshold → hi, else lo
latt	– (a values list)	snap to the nearest of the list
count	steps N before wrap	counter of clock edges: staircase lo-hi

Details:

- **tsh** (trigger sample-and-hold): an edge is the transition of clock from < 0.5 to ≥ 0.5 . Between edges the last sample is held. Combined with the `~win` window signals it implements onset triggers (chapter on windows as signals).
- **cmp**: the output is binary in range terms (lo or hi). The default threshold is the middle of the range. The comparator is the basic way to derive a gate from a continuous signal.
- **latt** (lattice): an arbitrary, possibly non-uniform lattice of values; the input snaps to the nearest element of the list. Unlike quantize, the lattice nodes need not be equal fractions of the range.
- **count**: on every clock edge the counter increments modulo N; the output is $lo + (hi-lo) \cdot cnt / (N-1)$. Wrapping around turns it into a polymetric stepper; the chain `count → cmp` gives a clock divider (a gate once per N edges).
- **fold**: the input is mapped through the range centre with gain and reflected off the bounds (triangular folding); the reflection period is two ranges.

Processor state (the current smoothed value, the held value, counters, phases) persists between ticks and is reset synchronously with a structural scene restart (chapter on reactivity).

Time units: an example

At 120 BPM one beat = 0.5 s. `lag` with amount 0.1 — a time constant of 0.1 beat = 50 ms; `s&h` with amount 4 — four samples per beat = 8 samples per second. When the tempo changes, all time constants rescale automatically — the patch keeps its musical, not absolute, geometry.

A numeric count example

`count` with amount = 4 ($N = 4$), parameter range 0...1. Consecutive clock edges produce: $0 \rightarrow 0.333 \rightarrow 0.667 \rightarrow 1 \rightarrow 0$ (wrap) $\rightarrow \dots$ Chained with `cmp` (threshold 0.9): the gate opens one edge out of four — a /4 divider. If clock is subscribed to a cell's `~win` (an edge on the onset, see the chapter on windows as signals), the staircase advances once per appearance of the cell: "once per 4 appearances" polymetry relative to the loop.

10 — ROUTING: PARAMETER OR INLET

A layer on the parameter name

A modulator with a parameter name adds a **layer** to that parameter's composition (chapter on composition). The range of effect is all sounding cells within the scope whose preset has the parameter. This is the default way for ramp and func; for a param lock it is the only way.

Writing into a target inlet (--> target)

Assigning a target switches the modulator into **writer** mode: its output ceases to be a layer and becomes a signal inside another module's input. The target is addressed by the modulator's stable identifier plus the inlet name; the addressing survives copying, moving and renaming of blocks. The target may be a modulator of any scope — parent, child, sibling, or the copy of a modulator in **another row** (chapter on cross-row chains).

Allowed target inlets: for source — from, to, rate, peak, susl, speed, hold, morph, phase, seed; for func — value, amount, clock.

Properties of writer mode:

- the modulator's kind is preserved: a func writer remains a stateful processor that happens to live "inside" a foreign input — this is how func stacks are built (e.g. dice → tsh → latt → parameter);
- the modulator's own parameter name stops being an assignment and remains only as the knobs' reference range;
- the write acts within the writer's window (the window contract); outside it the target rests on its own inlet value;
- cycles are allowed: mutual writes $A \rightarrow B$ and $B \rightarrow A$ are broken by a one-tick delay (z^{-1}) on the back edge.

Evaluation order

The modulator graph is sorted topologically over the edges "writer → target" and "subscription to a local output"; back edges of cycles read the previous tick's value. The order is deterministic.

11 — VALUE COMPOSITION

Layers on a parameter

A cell's final parameter value is assembled as follows:

1. **Base** — the cascade of `#`-channels from the cell up to the preset.
2. **Layers** of all modulators whose scopes lie on the cell's path (plus the preset layer), sorted by rank: senior first, junior later.
3. Each layer applies with its **combine** mode:

MODE	SYMBOL	SEMANTICS
over	over	replace the accumulated value (base-reactive*)
sum	+	add
mul	×	multiply (ducking/sidechain)
min	min	element-wise minimum (hard ducking)
max	max	element-wise maximum (ceiling)

* The over mode is "base-reactive": a live shift of the knob moves the centre of modulation — the override replaces the value yet follows the change of the base relative to the point it was computed against.

4. Layers with non-over modes (`+`, `×`, `min`, `max`) count as sidechain layers and apply **on top of** all over layers regardless of scope rank.
5. The result is clamped to the parameter's range (if a range is declared) and published into the cell's `=`-channel.

Inlet fan-in

When one inlet receives several writers, the same rank rule applies: writers are ordered by scope rank (ties — by declaration order); over-writers reduce by "last wins" — that is, **the deepest scope**; writers with `+`, `×`, `min`, `max` modes apply on top of the result in the same rank order. In addition to writers an inlet may carry a channel subscription (chapter on listen): the channel is applied over the inlet's own constant with its mode; in-patch writers take precedence over the subscription.

The uniformity is deliberate and worth stressing: **parameter layers and inlet fan-in are resolved by one and the same rank logic**. Learn the rule once and you can predict the behaviour of any point of the system.

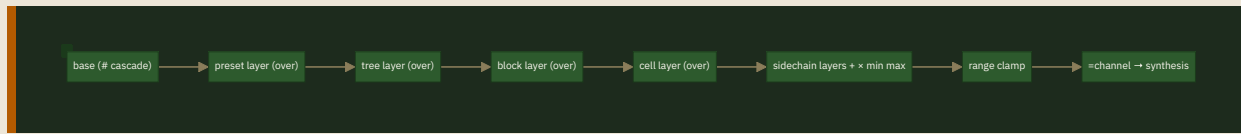


FIG. 2 – PARAMETER COMPOSITION: RANKS, THEN SIDECCHAIN, THEN CLAMP

A numeric composition example

Parameter `grPos`, base 0.20. Layers: a preset lag gate (over, output 0.30); a block ramp (over, value 0.55); a block dice jitter (`+`, value +0.07); a ramp on cell c:3 (over, value 0.10).

For cell c:3 inside its window: 0.20 → gate 0.30 → block ramp 0.55 → cell ramp 0.10 (junior rank overrides) → sidechain layer +0.07 → final 0.17. For the other cells (the c:3 window is closed): ... → block ramp 0.55 → +0.07 → 0.62. Note that the `+` layer landed on top of the over chain's result regardless of the scope it was declared on.

Fan-in: if both the block ramp and the cell ramp are routed into the gate's `value` inlet (rather than as layers), the rule is the same — inside the c:3 window the inlet reads 0.10, outside it 0.55; the gate itself remains the only layer on the parameter.

12 — REVERSE PATCHING: LISTEN

A point write ($\rightarrow$ target) is a patch from the **writer's** side: "my output goes into someone's input". A listen subscription is a patch from the **reader's** side: "my input listens to a channel". Together they form a complete patch bay.

A subscription is declared on an inlet and consists of a source selector, an attenuverter and a combine mode:

```
1 inlet ← source × depth + off (mode: over | + | × | min | max)
```

Source selectors

SELECTOR	CHANNEL	PURPOSE
m:1 ... m:4	m:<k>/#value	the global macro knobs
win	<own scope>/~win	own window phase
win:up	<parent>/~win	the parent window's phase
@<uid>	<target scope>/@<uid>	live output of another modulator
=<param>@<cid>	<cell>/=<param>	a cell parameter's final value (sidechain by value)

Selectors are symbolic: they resolve into concrete paths at session build, so subscriptions survive scene copying and block moves.

Semantics

- The source value is scaled by the edge: `src · depth + off`. The depth/off pair is a full attenuverter (e.g. depth -1, off 1 inverts the window phase).
- The result is applied over the **inlet's own constant** with the chosen mode; over means replacement.
- If the source channel is absent (the writer's window is closed, the modulator was deleted, the row is not playing) — the inlet silently falls back to its own constant. The window contract holds for subscriptions too: a subscription cannot "remember" the last value of a vanished source.
- A `@uid` subscription to a modulator of one's own graph participates in the topological sort (the current tick's value is read); a subscription to a channel of a foreign row reads the value published by that row's stream.
- The `=<param>@<cid>` subscription turns the **final value** of any parameter of any cell into a modulation signal: value sidechains, follower patterns, inter-parameter dependencies.

Both $\rightarrow$ target assignments and listen edges are part of the scene's

structural signature: adding or removing one is a structural edit (a smooth restart at the iteration boundary), whereas turning depth/off is a value edit (instant, no restart). See the chapter on reactivity.

13 – WINDOWS AS SIGNALS: CLOCKS AND ONSETS

Every open scope publishes the channel `<scope>/~win`, whose value equals the window phase ($0 \rightarrow 1$) and which exists exactly while the window is open. The temporal structure is thereby itself a signal source: no separate "clock generator" is needed.

Typical uses:

- **Window phase as a modulator.** The subscription `inlet ← win` yields a linear $0 \rightarrow 1$ signal within the bounds of one's own window; `win:up` — within the parent's (useful when a node wants to know "where are we in the phrase").
- **Onset trigger.** A tsh processor with the subscription `clock ← win`, depth -1 , off 1 : the inverted window phase equals 1 at the moment of opening and decreases; the upward crossing of 0.5 happens exactly at window open — tsh samples its input at the scope's onset (every slice, every branch entry). Without the inversion the edge lands mid-window.
- **Event counting.** A count with the same clock counts window **appearances** (one opening per loop for a block; one per branch play for a branch container), forming slow polymetric staircases "once per N appearances".
- **Fast clocks.** A square lfo (morph 1) with rate N , routed into a processor's `clock`, gives N edges per window — a dense internal clock for tsh/count. The speed of that clock is itself an inlet and can be modulated.

The phase inlet: time as an input

Every form has a `phase` inlet that **replaces** the window phase (wrapped into $[0, 1]$). This turns time from a given into a patchable signal:

SIGNAL IN PHASE	EFFECT
<code>constant c</code>	freeze the form at point c (knob scrub)
<code>driver $0 \rightarrow N$</code>	ratchet: N full form cycles per window
<code>driver $1 \rightarrow 0$</code>	reverse: the form reads backwards
<code>arbitrary curve</code>	warp/groove: nonlinear flow of time
<code>m:<k> (subscription)</code>	manual scrub of form position by macro knob

Ratchets and reverses hung on tree-branch modulators engage exactly when the branch is chosen — the window contract combined with phase patching (see the recipes).

14 — RANDOMNESS: RAND, DICE, VARIANTS

The determinism principle

All random quantities of the system are functions of an explicit seed. The base iteration seed is a hash of the tuple `(row, scene, iter, epoch)`. From it are independently derived: the iteration's branch choice vector; the dice deal of every modulator (with the modulator's identifier mixed in — distinct dice have distinct patterns). Consequences:

- the interface, the engine and the audio player agree on the choice with no synchronizing messages;
- any "random" behaviour is reproducible: the same iteration of the same epoch gives the same sound;
- predictability ahead: the choice and the deal of the next iteration are known in advance.

Branching

`rand` / `wrand` containers make one choice per branching point per iteration. Different modulators on different branches are the primary means of modulation variety: the branch arrives together with its patch.

`wrand` weights are numeric properties of the tree; additionally, at roll time the engine reads the channels `<branch>/#weight` (when they exist) — a live weight channel overrides the static one. Choices of already-announced iterations are frozen and not re-played.

The seed as an inlet

The `seed` inlet of the dice form replaces the iteration seed:

- **pin**: a constant seed fixes a deal you like — the pattern stops re-dealing while its neighbours keep re-dealing around it;
- **scrub**: the subscription `seed ← m:<k>` with a large depth (say 4096) turns a macro knob into a deal browser; every step of the knob is a new but deterministic deal.

15 – SPAN: MULTI-LOOP FORMS

The property `span = N` stretches a form's life cycle over N loop iterations: the effective form phase is $t = ((\text{iter} \bmod N) + \text{wp}) / N$. The form flows **across loop boundaries**, continuing from where it stopped; slow arcs of 2–16 loops are built with a single knob.

For dice, span also means the pattern re-deals once per N iterations (the seed is taken from the group number $\text{floor}(\text{iter}/N)$): the pattern "lives" N loops, then deals anew.

16 – MACRO CHANNELS

`m:1 ... m:4` are four global knobs. Each writes the channel `m:<k>/#value` in the shared space: one knob is instantly visible to the inlets of all modulators of all rows subscribed to it via listen. Macro values are stored in the save and re-published on every deploy/engine connection. Writing a macro is a value edit: no restarts.

What a macro does is defined solely by its subscriptions: the same knob can simultaneously drive jitter depth in one row, a gate threshold in another and a scrub position in a third.

17 — CROSS-ROW CHAINS

Presets are global, and a copy of every preset modulator exists in every row where the preset is used. A writer (→ target) aimed at a preset modulator acts through two paths at once:

- **in its own row** — by writing directly into the local copy's inlet;
- **in the other rows** — through a channel: the inlet of the copy in a foreign row is bound to the writer's output channel, keeping its own constant as the fallback.

The window contract works across row boundaries: the writer's channel exists exactly within its window, so the target in the foreign row is overridden (or combined — per the writer's combine) for exactly that window and returns to its own value outside it. The rows remain polymetric — the writer's window is defined by its own loop.

Together with the `=<param>@<cid>` subscription this yields cross-row value sidechains: a cell parameter of one row (after all lattices and layers) drives a modulator of another row.

18 — REACTIVITY AND LIVE EDITING

The system distinguishes two streams of edits:

- **Value edits** — knobs, from/to/rate, weights, subscription depth/off, macro values, lattice lists, span. Implemented as channel writes or hot description swaps: applied **immediately**, nothing restarts, no dice are re-rolled, func node state is preserved.
- **Structural edits** — adding/removing a modulator, cell or branch; changing the tree shape; creating/removing a $\rightsquigarrow$ target or a listen subscription. Applied **smoothly**: the new structure enters at the iteration boundary of the affected scenes; `epoch` increments (fresh dice), func node state and output channels reset synchronously with the boundary.

Affectedness is computed over the link graph: a chain or subscription between scenes makes both affected (transitively) — linked scenes restart together, preserving mutual consistency.

The distinguishing criterion is the scene's **structural signature**: the composition of the tree, the set of modulators and their links, without numeric values. Signature unchanged — hot swap; changed — smooth restart.

The practical consequence: fine-tuning the sound (values) never

interrupts the stream; re-wiring the patch (links) always enters musically — on the loop boundary.

19 – WORKFLOW AND THE PROJECT

The working cycle

F2 is built around a tree-based pattern sequencer compiled into SuperCollider's JITLib patterns. The core cycle: build a tree of blocks → F2 compiles it into SC code → SC plays it live. Everything is hot-swappable: value edits apply instantly, structural edits — on the loop boundary (chapter on reactivity).

Creating a project

The **Welcome** tab: create or select a project. A project is a folder on disk (structure below). F2 automatically loads SuperCollider and the project's synthesizers, effects and samples; the boot button starts the server, the boot log lives in the shell tab. The `ctx` button in the header re-reads the context (synth and preset definitions) without a restart.

Project folder layout

```

my-project/
├── Synths/                               SynthDefs — auto-loaded at boot
│   ├── synthPrimitives.scd
│   ├── kit1.scd
│   └── granuS.scd
├── Effects/                             effects — auto-loaded into the rack repository
│   ├── lushRev.scd
│   ├── repeater.scd
│   └── amp.scd
├── Samples/                             audio files (wav/aiff/flac) → the ~samples dict
│   ├── kicks/
│   └── textures/
├── ModalSamples/                         modal-analysis JSON → ~modalBuffers
└── Sessions/                             auto-saved JSON sessions
  
```

The boot file and the DSL are constructed automatically — placing them in the project by hand is not required (early versions demanded it).

Running from source

Dependencies: SuperCollider 3.13+, Rust 1.82+, Node.js 22+ (Linux additionally: webkit2gtk 4.1, libappindicator3, librsvg2, patchelf). Commands: `npm install` — frontend dependencies; `npm run tauri dev` — development mode with hot reload of the frontend and the Rust backend; `npm run tauri build` — distributable build.

The audio-server binaries are located automatically; non-standard paths are set through environment variables:

VARIABLE	DEFAULT	PURPOSE
F2_SCLANG	auto	path to the slang binary
F2_SCSYNTH	auto	path to the scsynth binary

20 — THE CANVAS: BLOCKS AND THE TREE

The canvas is the structure editor of one scene: its blocks and the tree that wires them.

Blocks

A block is a rectangular matrix of cells, w wide (time slots) and h tall (parallel lanes). Every cell carries: a preset index, a density (0 — pause: a time slot with no event and no window), an optional rhythmic pattern, its own modulators and locks.

- Width and height are set with the sliders of the block section in the sidebar or with palette commands (`width 1/2/4/8/16`, `height 1...8`).
- With $h > 1$ the block unfolds into a parallel of lanes: every lane is an independent sub-block in time; a cell is addressed by coordinates (x, y) .
- Selecting a block/cell on the canvas defines the content of the scope sidebar.

The tree

The tree is assembled from containers (operators `~q ~w ~c ~I ~IW`, chapter on time) and block references. Node properties: the operator, the weight (participating in the parent's `~w` / `~IW`), its own modulators and locks. One block may be referenced from several places of the tree; each reference forms its own scope path and its own windows, yet shares the block content (to modulate copies independently, use block copies — each has its own identifiers).

The cell context menu

Right-click on a cell: create a `param` (lock constant), `ramp`, `lock-ramp`, an oscillator (created as a ramp of the lfo form), an fx lock — all on the cell scope; jump to the cell section of the sidebar.

21 — THE PALETTE AND PRESETS

The right sidebar. On top — preset tiles: colour, click — select, shift-click — multi-select, right-click — edit colour, the «+» tile — new preset. Below — the editor of the selected preset:

- **Parameter knobs.** All numeric parameters of the instrument. A knob with an indicator dot is being modulated right now (a live ghost repeats the trajectory). Clicking a knob's label opens the parameter inspector and the LockGraph.
- For sample-based instruments the waveform of the loaded sample is shown.
- **The param locks section** — preset modulators: the preset scope bus, creation buttons (`+fx` , `+ramp` , `+fx ramp` , `+lock ramp` , `+ func`) and the cards. Preset func gates declared here are available as `→` target destinations from any scope of every row.

The preset is global: editing a preset or its modulators acts in

all rows and scenes where the preset is used. Preset modulators get a copy in every row; the copies are linked into cross-row chains automatically (chapter on cross-row chains).

22 — THE AUDIO ENGINE AND LIVE STATE

The bridge

Modulation is executed by a dedicated engine (a sidecar) connected to the interface over WebSocket and to the audio server over OSC. The bridge comes up automatically at application start; its state is shown by the `mod` menu in the header.

- **Bridge connected** — the engine drives all `=`-channels: modulation, ghosts, win-LEDs and live bus values are active.
- **Bridge absent** — sound runs on base values (the cascade without modulation); the interface stays fully operational, live indicators are inactive.
- **Version mismatch** — on connection the sides exchange a protocol version; a mismatch (a stale engine build) is reported with an explicit dialog. Continuing is possible but unsupported.

Live indicators

All live elements of the interface feed from a single ghost stream (~30 Hz, changes only):

INDICATOR	SOURCE	MEANING
dot on a parameter knob	cell <code>=channel</code>	the parameter is modulated now
knob ghost	<code>=channel</code> / driver output	live value trajectory
card win-LED	presence of the <code>@channel</code>	the modulator window is open
live value of a bus chip	<code>=channel</code>	the parameter's current result
value-stack rows	writers' <code>@channels</code>	contribution/"window closed"
live trace (LockGraph)	node <code>@channels</code>	output trajectories

An indicator having no value always means one of: the engine is not playing; the window is closed; the branch is not chosen. The bus window band (whether the scope sounds at all) and the win-LED (whether it is open right now) help tell these apart.

Launching scenes

A scene cell launch enters on quantization (scenes panel); the structure is deployed to the engine at launch and on edits (chapter on reactivity). A direct deploy from the editor (audition) sounds with base values, without the modulation engine.

23 — THE INTERFACE: OVERVIEW

The main window consists of the central canvas and slide-out panels. The left panels are mutually exclusive (scope / scenes) and open over the canvas; on the right slides out the palette.

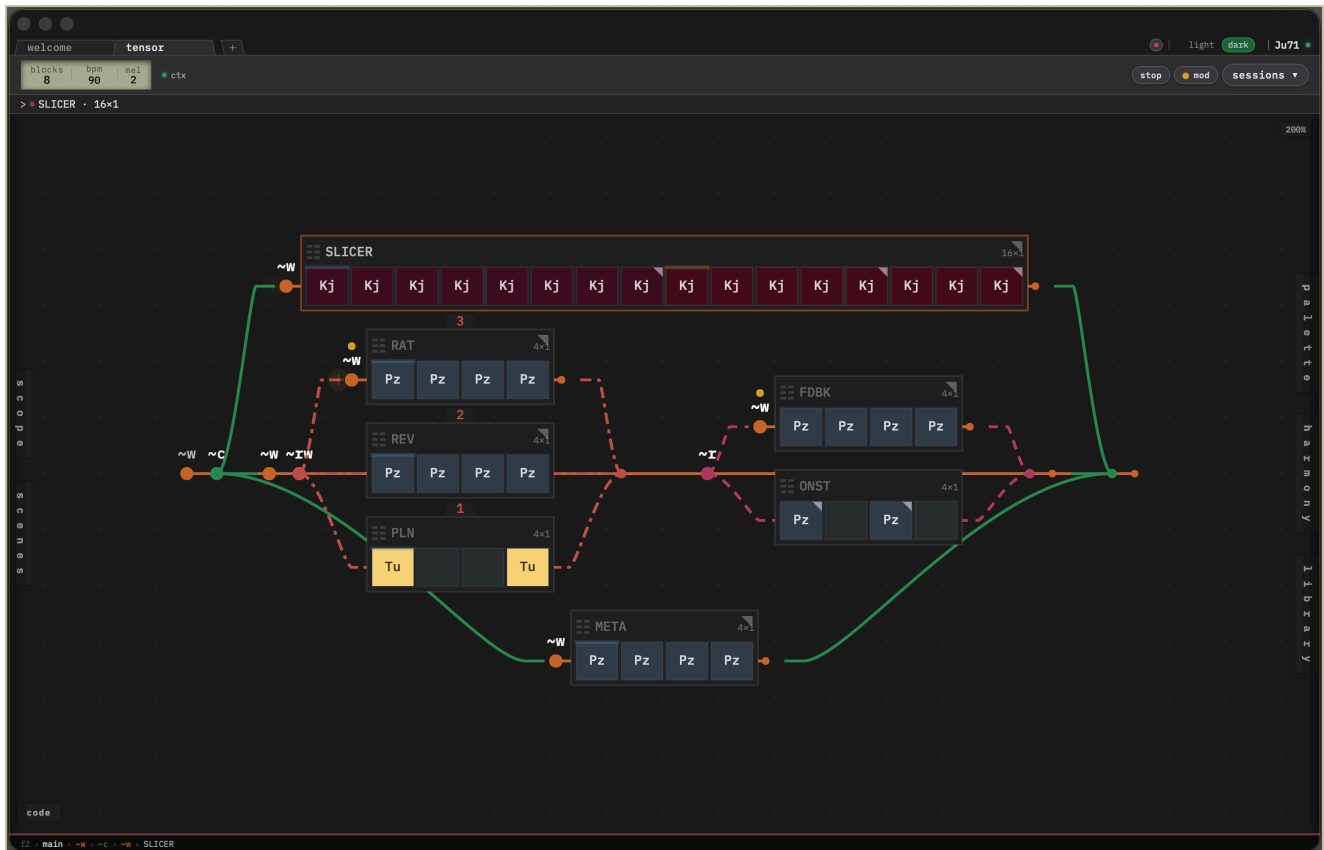


Fig. 1 — Main window overview

1. **Header.** On the left — the LCD block: block count, the `bpm` field (editable), the melodic tree counter (when harmony is on); the `ctx` indicator — the LED of the loaded synth context, click reloads the definitions. On the right — the `stop` button (silence everything), the `mod` menu (modulation engine state: enable/disable/restart, number of active slots), the `sessions` menu (saving and loading), the panel toggles.
2. **Canvas.** The grid of blocks and the structure tree. A block is a rectangle of cells; the tree wires blocks with the containers `~q ~w ~c ~r ~rw`. Selecting a block, cell or tree node defines the content of the scope sidebar. Right-click on a cell opens the modulator creation menu.
3. **The scope sidebar (left).** Modulator sections of the selected element: cell → block → container (top to bottom, when selected). Each section: the scope bus, creation buttons, modulator cards.
4. **The scenes sidebar (left).** The scene grid by rows; at the bottom — the macro knob strip. The `scenes | program` toggle in its header switches between the manual launch grid and the program constructor.

5. **The palette (right).** The preset list (colour tiles), the editor of the selected preset: parameter knobs, the `param locks` section (preset modulators) with its own scope bus.
6. **LockGraph.** The floating graph panel of the selected parameter (opens on knob selection; see the inspector chapter).
7. **Command palette.** Fuzzy search over parameters, locks and actions; contextual commands for block width/height and node weights.

All keyboard bindings are remappable in the Keymap editor (header

menu). The default bindings are collected in a table in the reference chapter; the authoritative list for a given install is the binding editor itself.

24 — MODULATOR CARDS

A card is the editor of one modulator. All three card kinds (`ramp` , `lock-ramp` , `func`) share one anatomy; the differences reduce to the knob set and the default routing.

Shared anatomy



Fig. 2 — Modulator card anatomy

1. **Title** — the kind and the current form/processor: `ramp dice` , `lock-ramp lin` , `func tsh` . On a ramp, clicking the title selects the parameter in the inspector.
2. **win-LED** — the window contract indicator: green — the modulator's window is open right now (its output channel exists), grey — the window is closed or the engine is not playing.
3. **The i button** — a popover with the card reference (a digest of the corresponding chapters of this manual).
4. **ParamPicker** — the "to a parameter name" assignment (the first routing way). The list of context parameters, grouped by family.

5. **→ target** — the "into a target inlet" assignment (the second routing way). Empty state — **→ target**; assigned — **→ target address**. Picking a parameter clears the target; picking a target dims the parameter pick. The cross next to the trigger clears the target.
6. **CombinePick** — the output combine mode: **over × min max +** (chapter on composition).
7. **Range / live value** — on ramp/lock-ramp the **from → to** text; on func — the live output (while playing).
8. **× (close)** — delete the modulator.
9. **The forms / processors row** — thumbnail icons of all curves (ramp/lock-ramp) or the chips of the nine processors (func).
10. **The knob row** — the parameters of the current form/processor (the exact sets are below).
11. **Preview** — the form at the current from/to values (the scale is the union of the parameter range and the actual bounds).
12. **+ listen** — the inlet subscription section (reverse patching; a separate section below).

Hovering a card highlights its counterparties with a dashed frame: the target, the writers of its inlets, the sources of its **@uid** subscriptions — whenever their cards are visible on screen.

The ramp card



Fig. 3 — Ramp card

The knob set by form (a knob is visible only when meaningful for the form):

Knob	Forms	Range	Meaning
from	all	parameter range	form start
to	all	parameter range	form end
rate	lfo rand dice	0.1 ... 32	periods/steps per window
morph	lfo	0 ... 1	sine → triangle → square
steps	step slew	2 ... 32	number of steps
speed	drift	0.5 ... 20	wander density
span	all	1 ... 16	stretch over N loops
peak	env	parameter range	attack peak
sus	env	parameter range	sustain level

The from/to knobs show a "ghost" — the live value when the inlet is driven by an external writer (the value arrives from its output; the knob itself then sets the fallback).

The lock-ramp card

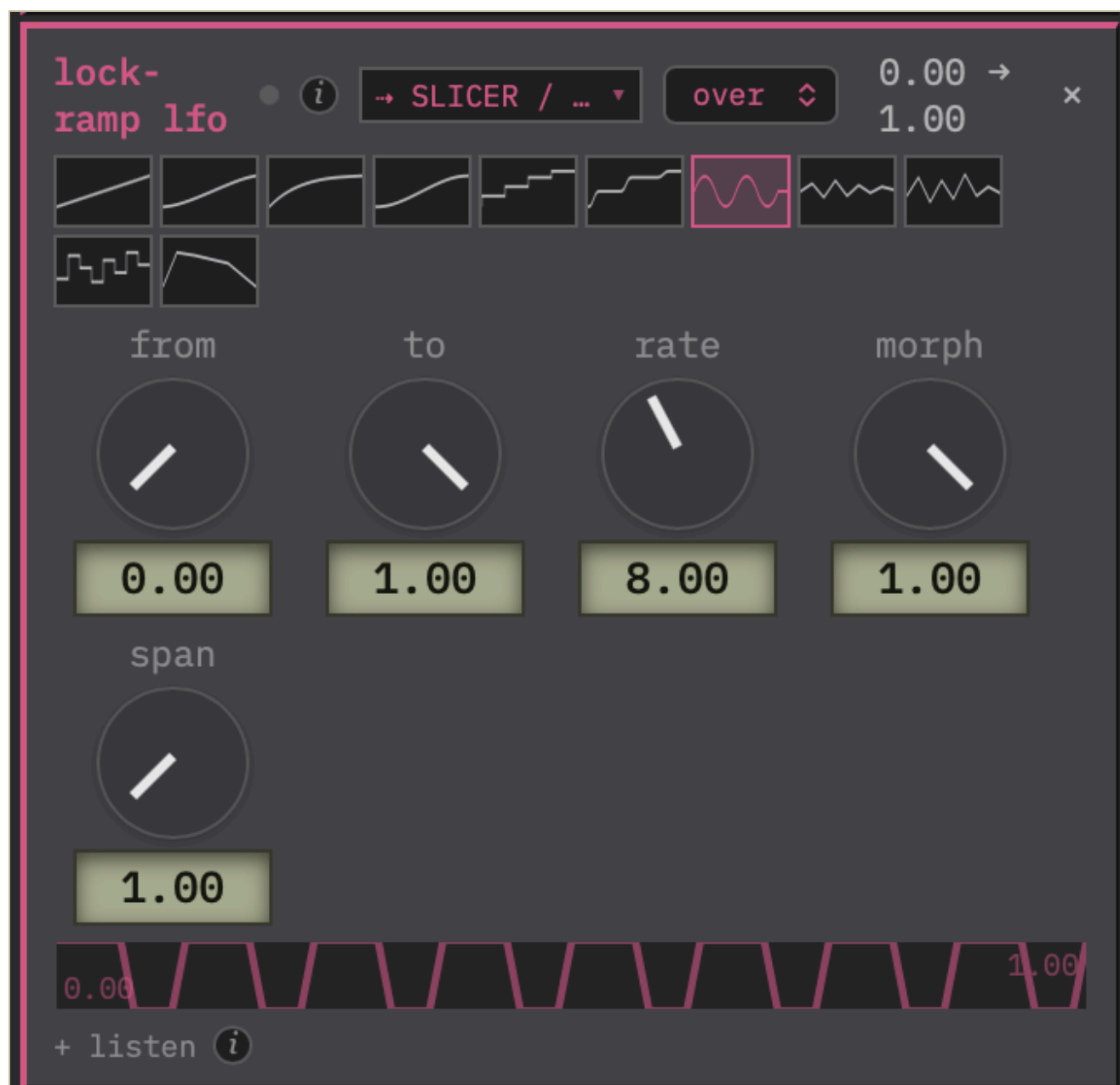


Fig. 4 — Lock-ramp card

Identical to ramp with these differences:

- the $\rightarrow$ target field is mandatory: the card body (forms, knobs, preview, listen) appears after a target is chosen;
- on target choice the from/to knobs receive values proportional to the target's current value ($\pm 50\%$), and their range — the range of the chain's root parameter (a walk down the chain of targets to the synth parameter);
- there is no ParamPicker: the parameter name is derived from the target and serves only as a range;

- knob ranges are somewhat narrower than ramp's: rate 0.1...20, steps 2...16 (the rest coincide).

The func card

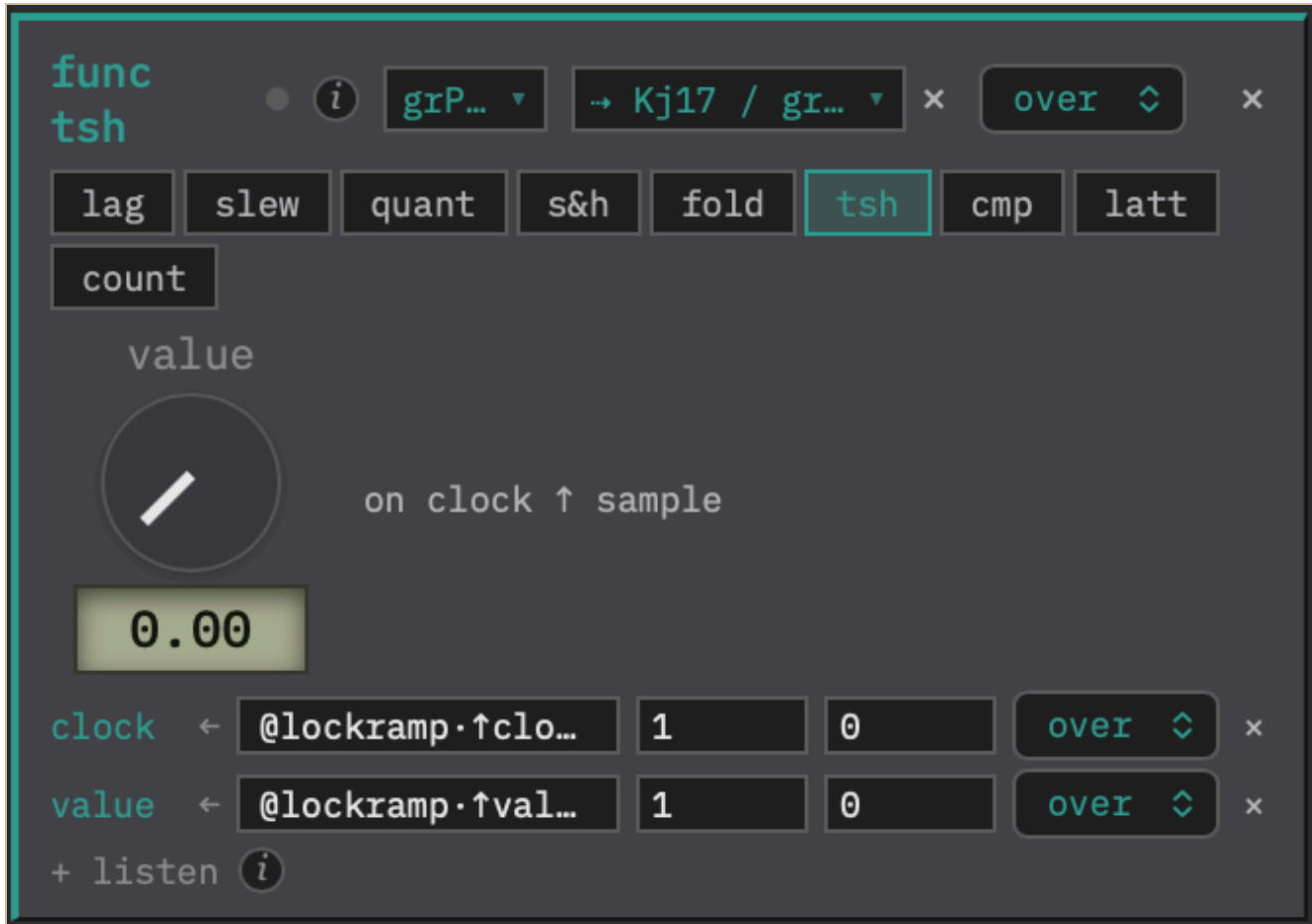


Fig. 5 — *Func card*

1. The processor row: `lag slew quant s&h fold tsh cmp latt count`. Switching the processor resets the amount knob to the new processor's default (the knob's meaning changes).
2. The **value** knob — the processor input (the value inlet's own constant); range — the parameter range.
3. The **amount** knob — the processor parameter; label and range depend on the processor (`time`, `rate`, `steps`, `gain`, `thres`); absent for `tsh` and `latt`.
4. The **lattice** field (only `latt`) — a comma-separated list of values; the input snaps to the nearest. Non-numeric entries are ignored.
5. The **on clock** ↑ hint (`tsh` / `count`) — a reminder that the processor is clocked by the edge of the clock inlet: either a → target into clock or the subscription `clock ← ~win`.
6. The live output — at the right of the title while playing.

The + listen section

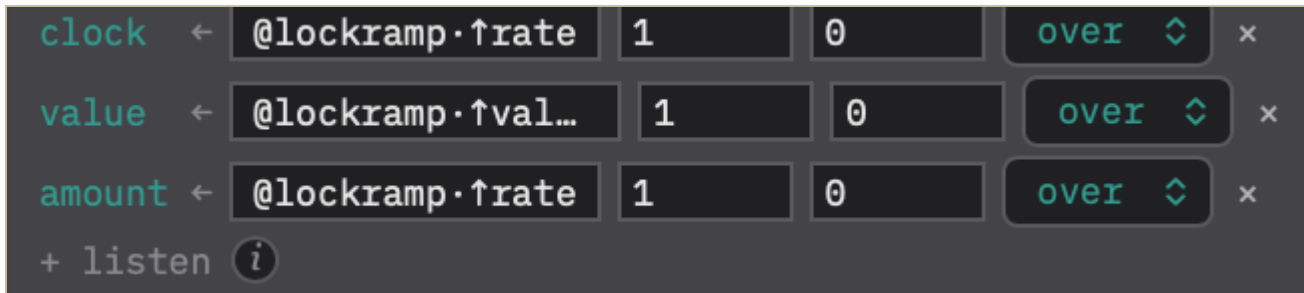


Fig. 6 — Listen section

Each line is one subscription:

1. **The inlet name** (in the card's accent colour).
2. The  arrow.
3. **The source** — the selector label (`m:2`, `~win`, `~win↑`, `@ramp.grPos`); the full selector in the tooltip.
4. **depth** — the source multiplier (numeric field, step 0.1).
5. **off** — the offset after the multiplier (step 0.05).
6. **The mode** — the CombinePick of application over the inlet's own constant.
7. **x** — remove the subscription.

The add line: the inlet selector (the inlet list of this card kind) → the source chips: `m:1...m:4`, `~win`, `~win↑`, then the live outputs of all modulators of the patch (`@kind.parameter`, full identifier in the tooltip). Clicking a chip creates a subscription with depth 1, off 0.

The target picker (→ target)

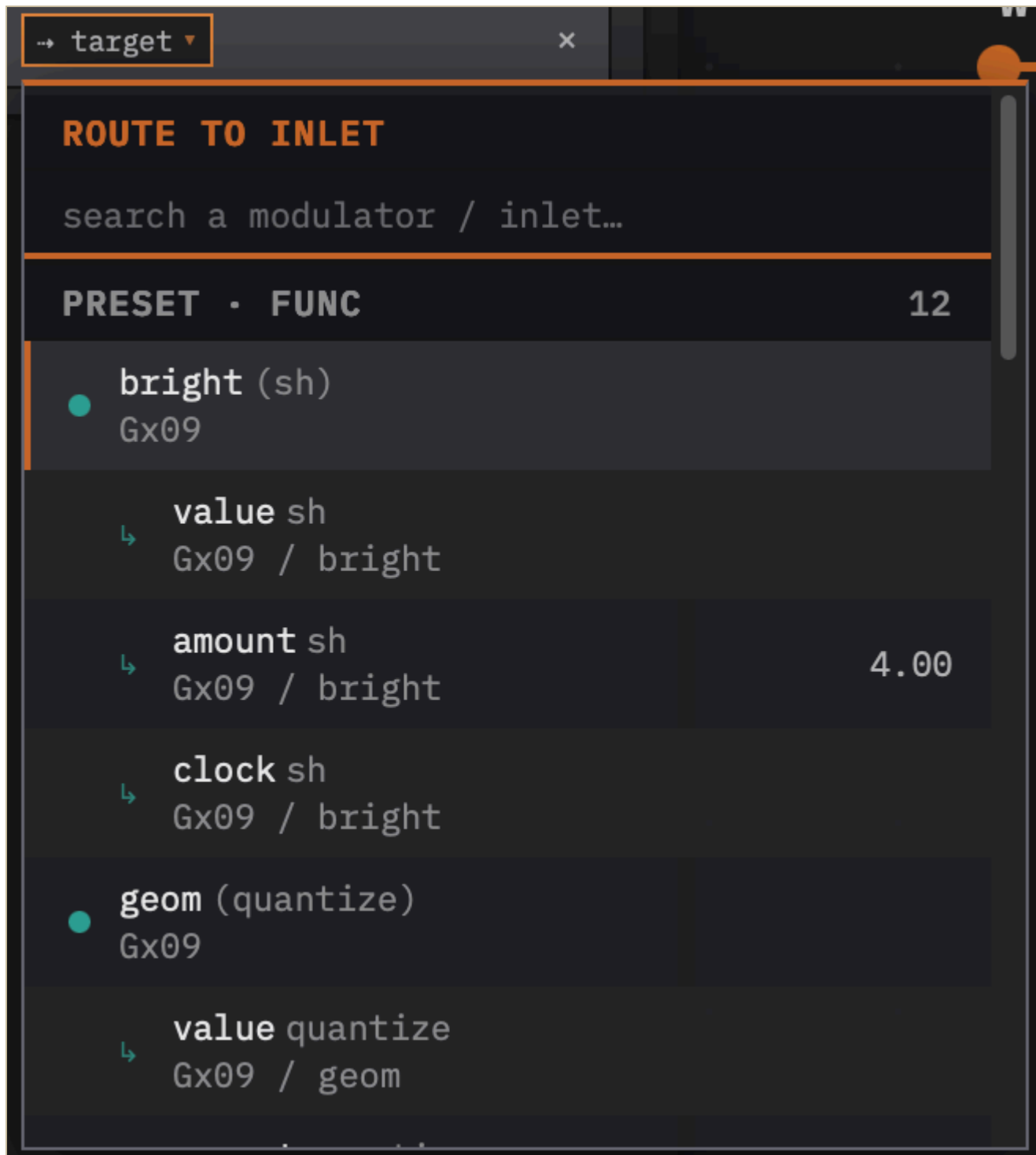


Fig. 7 — Target picker

The `route to inlet` window with a search field. Targets are grouped by scope with full addresses (block, cell coordinates, preset); an expanded modulator target lists its inlets as sub-items (for forms — from/to/rate/peak/susl/speed/hold/morph/phase/seed, for func — value/amount/clock). The current tar-

get is highlighted. The card's own self is excluded from the list (the self-loop is forbidden at the picker level; cycles through a third module are allowed and broken by z^{-1}).

25 – THE SCOPE BUS AND THE VALUE STACK

The bus (scope bus) is the first row of every modulator section; it displays the scope's assignments and signals and opens the value stack.

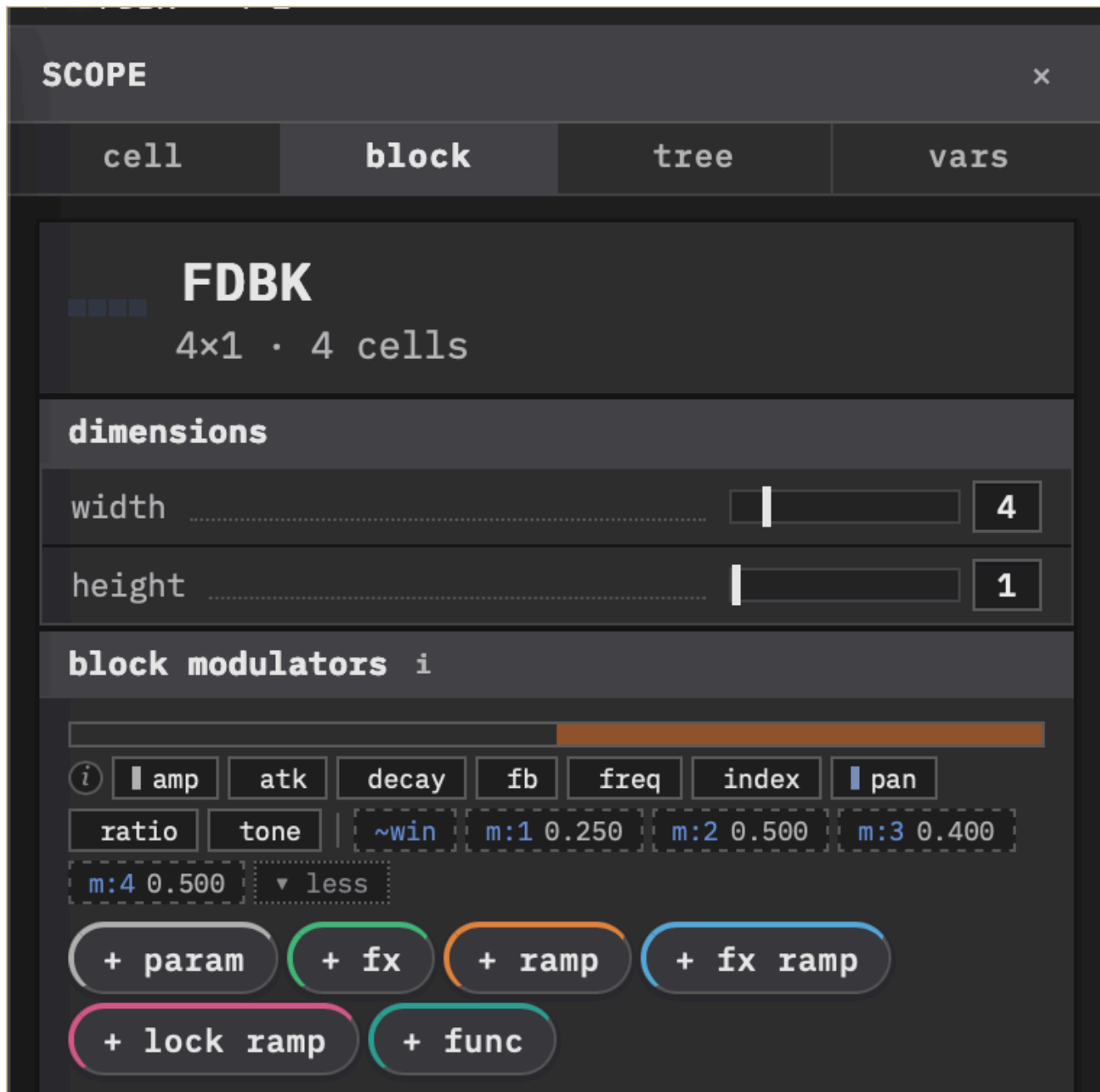


Fig. 8 — Scope bus

1. **The window band.** A ribbon of the row's loop with the window intervals of the current scope, computed from the phase tree. An opaque interval — the window exists in all variants; a translucent one — only in part of the rand branches (share of variants = opacity). The label `no window` — the scope

does not sound in the current tree. The preset section has no band (a preset's window is defined by its cells).

2. **The i button** — the bus reference.
3. **Chips of modulated parameters.** Collapsed, only parameters that already carry modulation are shown. The ticks on the left are the modulation scopes in rank order; colours: gold — preset, green — tree, blue — block, red — cell. The green number is the live value (while playing).
4. **no modulation yet** — the scope has no modulated parameters.
5. The **+N ▶** / **▼ less** button — expand all parameters and signal chips / collapse. The expanded list is height-limited and scrolls.
6. **Signal chips** (expanded): **~win** and **m:1...m:4** with live macro values — the list of channels available to listen subscriptions.

The value stack

Clicking a parameter chip opens the stack panel — the complete cascade of one parameter:

```

grRate - value stack Kj17

base #grRate 0.500

PRESET GATE · LATTICE GATE — INLET WRITERS, RANK ORDER
| chain · cell 15,0 → value over window closed
→ gate out —

SCOPE LAYERS · RANK ORDER (DEEPEST WINS)
| func lin · block over window closed
= live final not playing

dimmed = the writer exists but its window is closed
right now — its channel is absent and readers fall back.
Same rank rule for layers and inlet fan-in.

```

Fig. 9 — Value stack

1. The title: the parameter and the carrier preset.
2. **base #param** — the base value (the **#**-channel cascade).

3. The **preset gate** block (for every preset func on the parameter): the processor label; the list of **its inlet writers** in rank order — kind · scope → inlet name, combine tag, live value; the `→ gate out` line — the gate output.
4. The **scope layers** block — the scope layers on the parameter in rank order, each with a combine tag and a live value.
5. `= live final` — the result delivered to synthesis (while playing).
6. A dimmed (struck-through) writer or layer line means: **the object exists, but its window is currently closed** — the channel is absent, readers are on fallback. It is the window contract displayed at a moment in time.

26 – THE SCENES PANEL AND MACROS

48 / 70

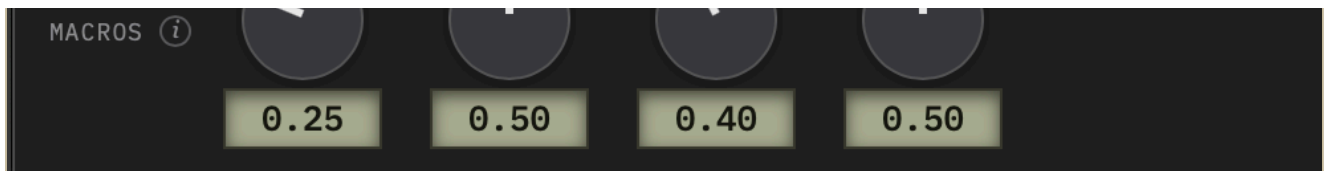


Fig. 10 — Scenes panel

1. **Rows.** Every row is a stream with its own loop. The coloured bar on the left shows `dur` (in beats); clicking opens a popover with a slider and a numeric field (2...128).
2. **Scene cells.** The square on the left — launch/stop (launch enters on quantization); the drag zone — reordering; double-click — load the scene into the editor. States: active (fill), pending (blinking until the boundary), edited (frame), empty (dim). A progress bar sweeps the active cell at loop tempo.
3. **The cell context menu:** `capture from editor` (capture the editor's current state into the scene), `copy / paste / duplicate`, `stop all`, `clear`.
4. **The `scenes | program` toggle.** The program mode replaces the grid with a launch-order constructor: per row a tree is built of scenes and the containers `~q` (sequence) and `~rw` (weighted random) with a repeat count per node; `END` sets the end policy (`∞` — from the start, `hold` — keep the last, `stop` — silence the row). A manual scene launch during a program puts the row into `MAN` (program paused).
5. **The macros strip** (bottom of the panel): the `macros` label, the `i` button, four knobs `m:1...m:4`. Turning writes the channel immediately; the values are part of the save.

27 — THE INSPECTOR, LOCKGRAPH, COMMAND PALETTE

- **Param inspector.** Opens on selecting a parameter knob (click on the knob label). Shows the selected parameter's layer stack by scope with operations: move a layer between scopes by dragging (↶), duplicate a layer, copy the stack, cut. It overlaps functionally with the bus stack; the inspector is oriented to editing, the bus stack to reading.
- **LockGraph.** The graph panel of the selected parameter: a vertical stack of layers with form thumbnails, the `live trace` button — display of live trajectories (node outputs in real time), clicking a layer selects its card. Driven inlets show the values of their drivers.
- **Command palette.** Search by name: parameters (jump to the knob), locks (jump to the card), actions. For a selected block — the `width` / `height` commands; for a tree node — `weight`.
- **Keymap editor.** The list of actions with their current combos; click — rebind, reset to default.

28 — THE EFFECTS RACK

The **Rack** tab is the editor of an effects tree of arbitrary complexity. The unit of the rack is the **bus**: a named stereo input into which presets are routed; the content of a bus is a pipeline of nodes. The **deploy** button compiles the tree into SC code and sends it to the audio server.



Fig. 11 — Rack tab

1. **Header.** LCD: the number of buses and the number of repository effects. Buttons: **save** / **load** — the rack configuration as JSON (stored separately from the session); **ctx** — re-read the project's Effects/ folder; **code** — the generated SC code panel; **deploy** — compile and send.
2. **The available fx repository.** All effects from Effects/; the LED marks the ones used in the tree.
3. **aliases.** The summary list of node aliases of the current tree.
4. **Buses.** Bus cards with pipelines; **+ IN bus** adds a bus; removal is available while more than one bus remains.

Pipeline nodes

- **fx** — an instance of a repository effect. The alias field is editable (default `<bus>.<effect>`); the alias is the node's name for fx locks. The chain order changes by dragging; **x** deletes the node.
- **merge** — mixes another bus's output into this point of the pipeline; the building element of send topologies (a send bus is processed by its own chain and merges into the common one).
- **split** — a branch into lanes (**a**, **b**, ...), each with its own sub-pipeline; lane outputs are summed at the node output. Modes: **crossover** — spectrum split at boundary frequencies (N lanes — N-1 boundaries), **copy** — full parallel copies of the signal.

Deploy and routing

Deploy generates one Ndef per node, wires the nodes with buses and orders the execution groups (sources → effect chains → master); the bus sum goes to the master. After deploy the context is re-read: the buses appear as values of the presets' **outBus** parameter — setting **outBus** routes a preset into a rack bus.

The deploy alias map is published to the fx lock cards: **fx** / **fx ramp** list the rack nodes by alias, and a node's parameters are the arguments of the effect function. Parameter values are smoothed on the rack side.

fx locks and fx ramps execute along the native audio path outside

the channel model: $\rightsquigarrow$ target routing and listen subscriptions do not extend to effect arguments (chapter on saving and compatibility).

29 — HARMONY

The **harmony** sidebar — the modal harmony tools: a chord progression, melodic voice trees and the binding of voices to palette presets. Harmony overrides only the pitch of the bound presets; their amplitude, timbre and all modulation remain intact.



Fig. 12 — *Harmony panel*

1. **chord progression.** The progression: chord tabs (root, quality, duration in beats), `+ chord` adds a chord, `x` removes. The `presets` button — a library of progression templates.
2. **The selected chord editor.** Root, quality, extensions; a circle of fifths with tendency arcs from the current root.
3. **voice trees.** Melodic trees; per tree — a set of voices. Voice properties: `rule` — how the voice picks a chord tone (`root` — the root, `degree N` — the Nth degree, `nearest` — minimal motion from the previous tone, voice leading), `pos` — register position, the octave range lo...hi, `slew` — portamento in beats, the preset set (several = unison/layers).
4. **The voicing preview.** Pitch lines [chord $\times$ voice] across the whole progression.

The number of melodic trees is shown in the main window header LCD.

30 – WRITING SYNTHESIZERS

A synthesizer is an `.scd` file in the project's `Synths/` folder; the file is executed at boot. It defines one or more `SynthDef`s and, optionally, registers presets in `~defs` and types in `~synthTypes`. After editing the file, `ctx` (re-read context) suffices — no server restart is needed.

The required contract

Every `SynthDef` declares the contract arguments:

```
1 SynthDef(\mySynth, {
2   |out=0, tel_bus=0, gate=1,
3   freq=200, pitchLag=0, amp=0.5, pan=0,
4   atk=0.01, dec=0.3, sus=0.7, rel=0.5, curve= -4,
5   // ... your custom params ...
6   colorID=1|
7
8   var env, sig;
9
10  // 1. PITCH – always use Lag for portamento support
11  var smoothFreq = Lag.kr(freq, pitchLag);
12
13  // 2. ENVELOPE – gate-based ADSR, doneAction: 2
14  env = EnvGen.kr(
15    Env.adsr(atk, dec, sus, rel, 1, curve),
16    gate, doneAction: 2);
17
18  // 3. YOUR SIGNAL CHAIN
19  sig = SinOsc.ar(smoothFreq) * env * amp;
20
21  // 4. TELEMETRY – call ~mkTel for visualizer data
22  //   args: synthTypeID, colorID, env, amp, pan,
23  //         posX(0-1), posY(0-1), p0, p1, p2, p3, tel_bus
24  ~mkTel.(42, colorID, env, amp, pan,
25    pan.linlin(-1, 1, 0, 1),
26    freq.explin(20, 20000, 0, 1),
27    0, 0, 0, 0, tel_bus);
28
29  // 5. OUTPUT – stereo via Pan2
30  Out.ar(out, Pan2.ar(sig, pan));
31  }).add;
```

ARGUMENT	PURPOSE
<code>out</code>	output bus; set by rack routing
<code>tel_bus</code>	telemetry control bus (provided by F2)
<code>gate</code>	required for pattern-based envelopes
<code>freq amp pan</code>	standard pattern keys
<code>pitchLag</code>	glide (Pmono support)
<code>atk dec sus rel curve</code>	standard ADSR (driven by patterns)
<code>colorID</code>	integer visualizer colour

Rules: `doneAction: 2` — the synth frees itself when the envelope ends; `~mkTel` carries real-time telemetry (the visualizer, the waveform monitor).

Registering presets

After the `SynthDef` the file may declare named presets — an instrument plus default parameter values:

```

1  ~defs[\myBass] = (
2      instrument: \mySynth,
3      atk: 0.001, dec: 0.25, sus: 0.3, rel: 0.4,
4      tone: 2500, drive: 1.3, colorID: 1
5  );
6
7  ~defs[\myPad] = (
8      instrument: \mySynth,
9      atk: 0.8, dec: 1.0, sus: 0.9, rel: 2.0,
10     vibDepth: 0.1, colorID: 6
11 );

```

Presets are imported on `ctx` and appear in the palette; every numeric parameter becomes a knob. Parameters are resolved automatically and are hot: a change while playing is heard immediately (chapter on reactivity); parameter names enter the ParamPicker lists, grouped by family.

Registering types (optional)

The synth class and signature parameters for the visualizer:

```
1 ~synthTypes = ~synthTypes ?? Dictionary.new;
2 ~synthTypes[42] = (
3   name: \mySynth,
4   class: \tonal,          // \tonal, \perc, \noise, \pad
5   params: #[tone, drive, vibDepth, modDepth]
6 );
```

A complete example: an FM synth with presets

```

1  SynthDef(\fmX, {
2    |out=0, tel_bus=0, gate=1,
3    freq=200, pitchLag=0, amp=0.5, pan=0,
4    atk=0.01, dec=0.3, sus=0.7, rel=0.5, curve= -4,
5    ratio=2, index=5, fbAmt=0,
6    drift=0, vibRate=5, vibDepth=0,
7    tone=16000, drive=1.0,
8    colorID=1|
9
10   var env, sig, smoothFreq, mod, carrier;
11
12   smoothFreq = Lag.kr(freq, pitchLag)
13     + (SinOsc.kr(vibRate) * vibDepth * freq * 0.01)
14     + (LFNoise1.kr(drift.linlin(0,1,0.1,5)) * drift * freq * 0.004);
15
16   env = EnvGen.kr(Env.adsr(atk,dec,sus,rel,1,curve), gate, doneAction:2);
17
18   mod = SinOsc.ar(smoothFreq * ratio) * smoothFreq * index;
19   carrier = SinOsc.ar(smoothFreq + mod + (LocalIn.ar(1) * fbAmt * smoothFreq));
20   LocalOut.ar(carrier);
21
22   sig = carrier * env * amp;
23   sig = LPF.ar(sig.tanh, tone);
24
25   ~mkTel.(10, colorID, env, amp, pan,
26     pan.linlin(-1,1,0,1), freq.explin(20,20000,0,1),
27     index/20, fbAmt, ratio/16, drive/10, tel_bus);
28
29   Out.ar(out, Pan2.ar(sig, pan));
30 } ).add;
31
32 // Presets
33 ~defs[\fmBass] = (instrument: \fmX, ratio:1, index:10, fbAmt:0.15,
34   atk:0.001, dec:0.25, sus:0.3, rel:0.4, tone:2500, colorID:1);
35 ~defs[\fmLead] = (instrument: \fmX, ratio:2, index:6,
36   atk:0.05, dec:0.3, sus:0.7, rel:0.6, vibDepth:0.08, colorID:1);

```

31 – WRITING EFFECTS

An effect is an `.scd` file in the Effects/ folder containing a **bare function** (not a `SynthDef`). F2 wraps the function into an `Ndef` inside the rack. The file name becomes the effect name: `myDelay.scd` → `\myDelay` in the rack repository.

The required signature

```

1  // Effects/myDelay.scd
2  { |in, mix=0.5, time=0.25, feedback=0.5, tone=8000|
3
4      var sig = In.ar(in, 2);
5
6      // the signal chain
7      var delayed = CombC.ar(sig, 2.0, time, time * 12 * feedback);
8      delayed = LPF.ar(delayed, tone);
9
10     // dry/wet – always by crossfade
11     XFade2.ar(sig, delayed, mix * 2 - 1);
12 }
```

Rules:

- the first argument must be `in` (a bus index); F2 validates the file as an effect by it;
- read stereo — `In.ar(in, 2)`; return a stereo signal (2 channels);
- dry/wet — `XFade2` or an equivalent;
- parameters are smoothed by the rack automatically (`Lag`); your own smoothing is unnecessary but allowed for special control;
- the function's arguments become the node parameters in the fx lock cards.

A minimal effect: volume and pan

```

1  // Effects/amp.scd
2  { |in, amp=1.0, pan=0.0|
3      var sig = In.ar(in, 2);
4      sig = Balance2.ar(sig[0], sig[1], Lag.kr(pan, 0.05));
5      sig * Lag.kr(amp, 0.01);
6  }
```

A reverb with safety

```

1  // Effects/lushRev.scd
2  { |in, mix=0.5, decay=6, damp=0.2, brightness=5000, tel_bus|
3      var sig = In.ar(in, 2);
4      var input = LeakDC.ar(sig);
5      var verb = FreeVerb2.ar(input[0], input[1], 1.0,
6          decay.linlin(0, 10, 0.5, 0.98), damp);
7      verb = LPF.ar(verb, brightness.max(100));
8      verb = Sanitize.ar(verb);
9      XFade2.ar(sig, verb, mix * 2 - 1);
10 }
```

A feedback delay

```

1  // Effects/repeater.scd
2  { |in, mix=0.5, time=0.25, feedback=0.5, tone=8000|
3      var sig = In.ar(in, 2);
4      var dt = Lag.kr(time.clip(0.005, 2.0), 0.01);
5      var fb = Lag.kr(feedback.clip(0, 0.95), 0.01);
6      var delayed = sig + LocalIn.ar(2);
7      delayed = CombC.ar(delayed, 2.0, dt, dt * 12 * fb);
8      delayed = LPF.ar(delayed, tone.max(200));
9      LocalOut.ar(delayed * fb);
10     XFade2.ar(sig, Limiter.ar(delayed, 0.95), mix * 2 - 1);
11 }
```

Always limit feedback loops (``Limiter.ar`` or ``.tanh``). Put

`Sanitize.ar` after reverbs — it catches NaN at extreme settings; put `LeakDC.ar` before them against DC build-up. An effect may declare a `tel_bus` argument — the `~mkTel` contract is the same as for synths. Parameter ranges can be declared with `Spec.add` when needed.

32 – SAVING AND COMPATIBILITY

A save is a single JSON (`f2_tensor`, version 3): presets, blocks, the editor tree, the rows' scenes (every scene cell carries its own copies of blocks and tree), parameter ranges (`paramRange`), macro values (`macros`), harmony data. All modulators carry stable identifiers; on load, missing identifiers are assigned and index references of old saves are migrated.

Compatibility notes:

- modulator fields unused by the current engine (artifacts of earlier versions) are ignored on load and do no harm;
- `paramRange` defines knob ranges, the delivery clamp and the lo/hi of func processors; with no range declared, a parameter is treated as 0...1;
- fx locks and fx ramps execute along the native audio path outside the channel model (→ target/listen routing does not extend to fx arguments).

33 — DIAGNOSTICS

Symptoms and the order of checks. All checks use the standard indicators; no external tools.

A modulator is not heard.

1. The window band of its scope's bus: is there a window at all (**no window** — the scope is not included in the tree); a translucent window acts only in part of the rand variants — wait for the branch or check the weights.
2. The card's win-LED: grey while the row plays — the window is closed right now (wrong phase interval / wrong branch).
3. The parameter's value stack: the modulator's line struck through — the window is closed; the line is there but a junior over sits above — the contribution is overridden by rank; no line at all — the modulator is routed into an inlet (→ target), not as a layer: look at the gate block.
4. A **+** / **×** combine with neutral values (0 for **+**, 1 for **×**) is indistinguishable from absence — check the from/to amplitudes.

A knob has no effect.

1. The knob is overridden by a junior scope (a cell lock over the block one, etc.) — look at the chip ticks and the stack.
2. The inlet is driven by a writer — the knob shows the driver's ghost and only sets the fallback: clear the writer's target or change its parameters.
3. The inlet is subscribed (listen) with the over mode — the channel exists and replaces the constant: remove the subscription or change the mode.

"Randomness" repeats every loop. The rand form is in use (the pattern does not depend on the iteration). For a fresh deal every loop — dice.

Dice does not re-deal. The seed inlet is occupied by a constant (a pin) or by a macro subscription. Remove the pin; with span > 1 the re-deal happens once per span loops — that is the declared behaviour.

Branch choice "ignores" weight changes. The choices of the current and the next iteration are frozen (already announced to the audio player); new weights enter from the iteration after the announced one. A structural tree edit re-rolls immediately (iteration boundary, epoch).

Ratchet/reverse has no effect. The phase driver lives on a scope whose window is closed (or the branch not chosen) — check its scope's window band; phase is replaced only within the driver's window.

Values "jump" at the loop boundary. A structural edit entered at the boundary (epoch, func state reset) — standard behaviour; value edits do not wait for the boundary.

Live indicators are empty. The **mod** menu: the engine is off or the bridge is down; or the row is not launched. A bridge version mismatch is reported by a dialog on connection.

34 — RECIPES

The format of every recipe: goal → patch (scope, modulator, settings) → behaviour. Parameter ranges are assumed 0...1 and the loop 32 beats unless stated otherwise.

Branch ratchet

Goal: repeat the reading of the main ramp N times, only when the branch comes up. Patch: the `~rw` branch container → `lock-ramp`, form lin, from 0, to 4 → the `phase` inlet of the block's main ramp. Depth control: the subscription `to ← m:1 × 8` (macro 0...1 → ratchet 0...8). Behaviour: within the branch window the ramp's phase runs 0→4 — four full cycles (wrapped); outside the window the driver does not exist and the ramp runs on its own time. A fractional to gives a partial last repeat.

Branch reverse

Patch: the branch container → `lock-ramp` lin from 1, to 0 → the target's `phase`. Behaviour: the target reads backwards exactly within the branch window. Combined with a ratchet on a sibling branch this gives probabilistic alternation of reading directions.

Stepped reading (slice-freeze)

Goal: the main ramp sounds as a staircase of N steps while keeping ratchets/reverses. Patch: block → `lock-ramp` lfo, morph 1 (square), rate 8 → the `clock` of a func `tsh`; the func `tsh`: subscription `value ← @main-ramp`; output → the `value` of the preset position gate (lag). Behaviour: tsh samples the ramp on every square edge — 8 steps per window. Since it samples the ramp's live output, any distortions of its phase (ratchet, reverse) pass through the sampler. The staircase speed is modulatable: the square's rate is an inlet (e.g. a meta-chain 4→12 over the loop).

Onset counter

Goal: a parameter climbs one step at every appearance of a scope, wrapping every N. Patch: a cell (a cell of a silent carrier block suffices) → `func count`, amount N, subscription `clock ← ~win × -1 + 1`; output → the target inlet (e.g. the `value` of a drive gate). Behaviour: the inverted window phase crosses 0.5 upwards at the window opening — one edge per appearance. Counting goes by appearances (for a rand branch — by its draws), wrapping every N: polymetry relative to the loop.

Clock divider

Patch: `count` (N steps) → the `value` of a comparator `cmp`, threshold 0.9, combine ×, on amp.

Behaviour: `cmp` outputs hi only on the last step of the staircase — a gate once per N edges of the input clock.

A per-slice lattice

Goal: the rate parameter takes only musical values, fresh every cycle. Patch: `preset` → `func latt`, list `0, 0.25, 0.5, 1`, on the rate parameter; `block` → `lock-ramp dice`, rate 16, from 0, to 1 → the lattice's `value`. Behaviour: 16 rolls per loop, each snapped to the nearest lattice node; the deal refreshes every loop. A brake at the end: `cell 15` → `lock-ramp` lin 0.5→0.05 → the same inlet — the cell rank overrides the block dice exactly on the last slice, and the brake also passes through the lattice.

Pinning and scrubbing a deal

Pin: on a dice, the seed knob = an integer — the deal is fixed while neighbouring dice keep re-dealing.

Scrub: the subscription `seed ← m:4 × 4096` — every position of the macro knob corresponds to a deterministic deal; turning browses the variants, releasing keeps the current one.

A cross-row sidechain gate

Goal: the bass of row B opens from the slicer rate lattice of row A. Patch: row A, `block` → `lock-ramp` lin with subscriptions `from ← =grRate@<cid>` and `to ← =grRate@<cid>` (from = to = the channel value ⇒ a constant output equal to the cell parameter's final value) → the `value` of a preset `cmp` on the bass amp; the `cmp`: combine ×, subscription `amount ← m:3`. Behaviour: `cmp` compares the live slicer rate (after the lattice) with the macro threshold and multiplies the bass amp by 0/1. It works across rows thanks to the shared channel space; the threshold is a performance knob.

A swell from a foreign row

Patch: a branch container of row A → `lock-ramp env` (peak to taste) → the `to` of a standing amp ramp of the bass preset (from = to = the base level). Behaviour: the copy of the preset ramp in row B receives the chain's channel exactly within the window of row A's branch — the bass rises on every draw of that branch and returns to base outside the window. The rows remain polymetric.

A multi-loop arc

Patch: `block` → `ramp sin`, span 4, on a timbre parameter; or `dice` rate 16 span 2 — the deal "lives" two loops, the pattern stretched over 32 steps.

Mutual modulation (controlled chaos)

Patch: container → A `lock-ramp lfo` rate 3 → the `to` of B; B `lock-ramp lfo` rate 5 → the `rate` of A; a tap: a third modulator with the subscription `value ← @B`, output to an audible parameter (e.g. the `value` of a comb-mix gate). Behaviour: the A↔B cycle is broken by z^{-1} — the values are finite and deterministic; the system breathes nonlinearly yet reproducibly. Audibility comes from the tap: the cycle participants themselves are assigned to nothing.

Freezing the position on one slice

Patch: cell c:12 → `lock-ramp` lin from = to = 0.2 → the `value` of the position gate. Behaviour: a junior-rank constant writer intercepts the position bus exactly on the cell's slot — a one-slice "lock" while the live ramp keeps moving around it.

35 — REFERENCE TABLES

Forms

FORM	PARAMETERS	PERIODICITY	ITERATION DEPENDENCE
lin	—	1 cycle/window	no
exp	—	1 cycle/window	no
log	—	1 cycle/window	no
sin	—	1 cycle/window	no
lfo	rate, morph	rate cycles/window	no
step	steps	1 cycle/window	no
slew	steps	1 cycle/window	no
drift	speed	aperiodic	no
rand	rate	steps/window	NO (fixed pattern)
dice	rate, seed, span	rolls/window	YES (re-deals)
env	peak, sus	1 cycle/window	no

Form inlets

INLET	FALLBACK	PURPOSE
from	0	form start
to	1	form end
rate	1	periods/steps per window
peak	to	env peak
susl	midpoint	env sustain
speed	4	drift density
hold	4	step/slew steps
morph	0	lfo shape
phase	window phase	time replacement (wrap 0...1)
seed	iteration seed	dice seed replacement

Channels and listen selectors

SELECTOR	CHANNEL	EXISTENCE
m:1 ... m:4	m:<k>/#value	always
win	<scope>/~win	within the scope window
win:up	<parent>/~win	within the parent window
@<uid>	<scope>/@<uid>	within the modulator window
=<param>@<cid>	<cell>/=<param>	always

Scope ranks and colours

SCOPE	RANK	UI COLOUR
preset	0 (senior)	gold
tree	by depth	green
block	deeper	blue
cell	junior	red

Keyboard (default bindings)

mod = Ctrl (⌘ on macOS). Everything is remappable in the Keymap editor.

KEY	ACTION
1	scope sidebar
2	scenes sidebar
3	palette
4	harmony sidebar
mod+K	command palette
mod+5	library panel
mod+6	LockGraph
mod+2 mod+3	radial pickers: scenes and presets
mod+S	save session
mod+Z	undo
mod+Shift+Z mod+Y	redo
mod+C X V	copy, cut, paste
mod+D	duplicate block
Backspace	delete selected block
Escape	close, deselect
arrows	cell navigation within a block
Space	cell: toggle on/off
Enter	fill cell with the current preset
[]	previous / next preset
, .	previous / next block
= -	cell density up / down

36 — GLOSSARY

- **Scope** — a node of the hierarchy (preset / tree container / block / cell), owner of modulators; defines a window and a rank.
- **Window** — the loop-phase interval in which a scope exists; computed at variant compilation.
- **Window phase** — normalized time inside a window, 0...1; the argument of every form.
- **Window contract** — the rule "a modulator acts exactly within its window; outside it its output is absent and readers are on fallback".
- **Variant** — a complete resolution of all random branchings of the tree for one iteration; a flat list of segments with precomputed windows.
- **Iteration (iter)** — the row's loop number since the start of time.
- **Epoch** — the counter of structural scene restarts; enters the seeds.
- **Channel** — a named value in the shared path space; may be absent.
- **Base** — the parameter value from the `#`-channel cascade, without modulation.
- **Layer** — a modulator's contribution to a parameter's composition.
- **Combine** — the application mode of a layer/contribution: over, +, ×, min, max.
- **Rank** — scope depth; the junior (deeper) overrides the senior.
- **Writer** — a modulator routed into a target's inlet ($\rightarrow$ target).
- **Subscription (listen)** — a channel source of an inlet with an attenuverter and a mode.
- **Gate** — a preset func on a parameter serving as the common entry point for writers of all scopes.
- **Deal** — a concrete dice pattern determined by a seed.
- **Pin** — fixing a deal with a constant seed.
- **Span** — stretching a form's life cycle over N iterations.
- **Macro** — a global knob-channel `m:k`, available to subscriptions of all rows.
- **Value stack** — the panel of one parameter's full cascade: base → gates → layers → final.
- **win-LED** — the open-window indicator of a modulator's card.
- **Segment** — a cell's interval in loop phase with the ready windows of every scope on its path.

F2 – user manual
